

PHIẾU BÀI TẬP THỰC HÀNH

Họ và tên: Nguyễn Văn Nam

MSSV: 23644171

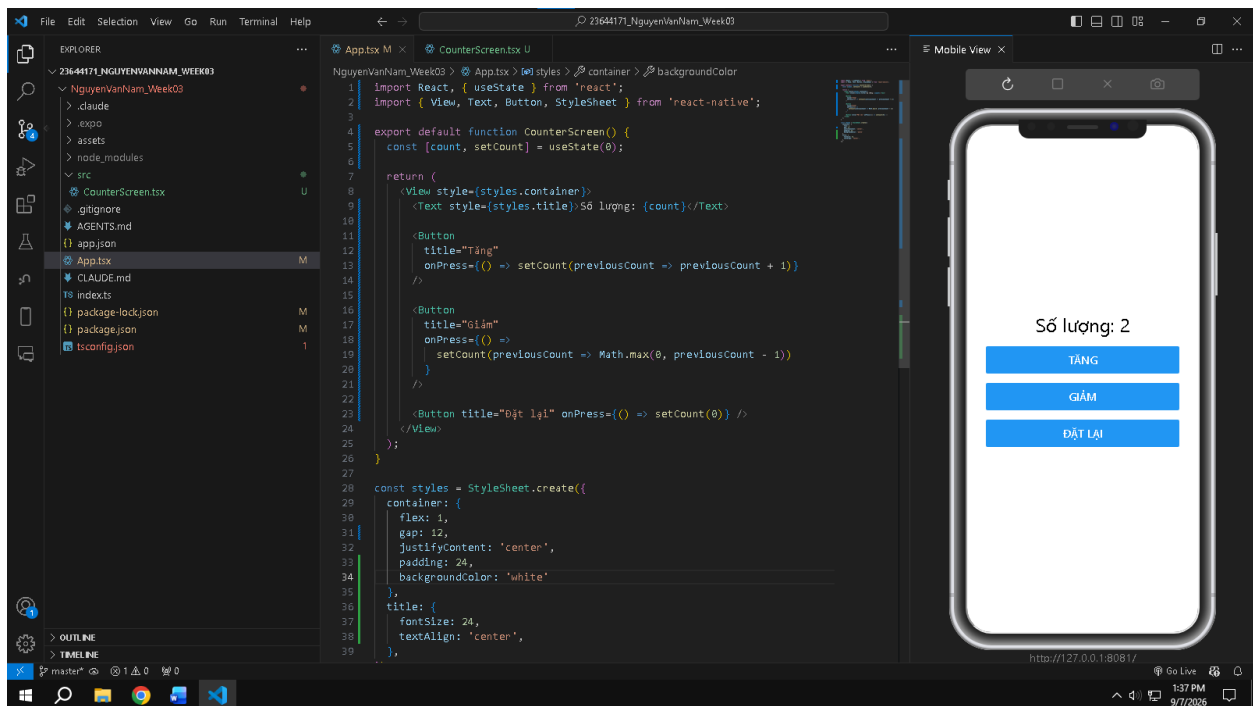
Link github chứa source code:

https://github.com/vannam25032005/ReactNative/tree/0f697a1018ad4cab11c4b2a0000d206a1ee14f45/23644171_NguyenVanNam_Week03

Chương 3. Hook trong React Native

3.1.1. useState

Ví dụ áp dụng: Bộ đếm số lượng



3. Bài tập thực hành: Form nhập họ tên

Yêu cầu: Tạo màn hình cho phép người dùng nhập họ tên và hiển thị lời chào.

Hướng dẫn từng bước

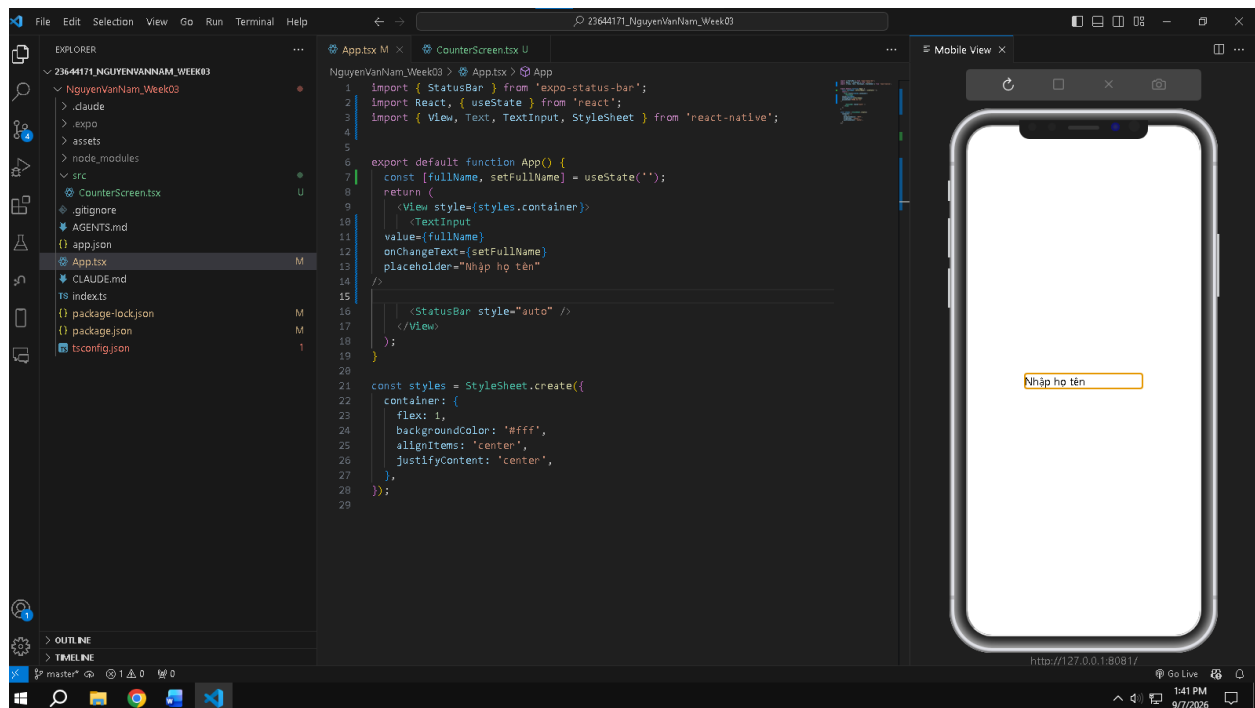
Bước 1: Import các thành phần cần thiết.

```
NguyenVanNam_Week03 > App.tsx > App
1 import { StatusBar } from 'expo-status-bar';
2 import React, { useState } from 'react';
3 import { View, Text, TextInput, StyleSheet } from 'react-native';
4
5
```

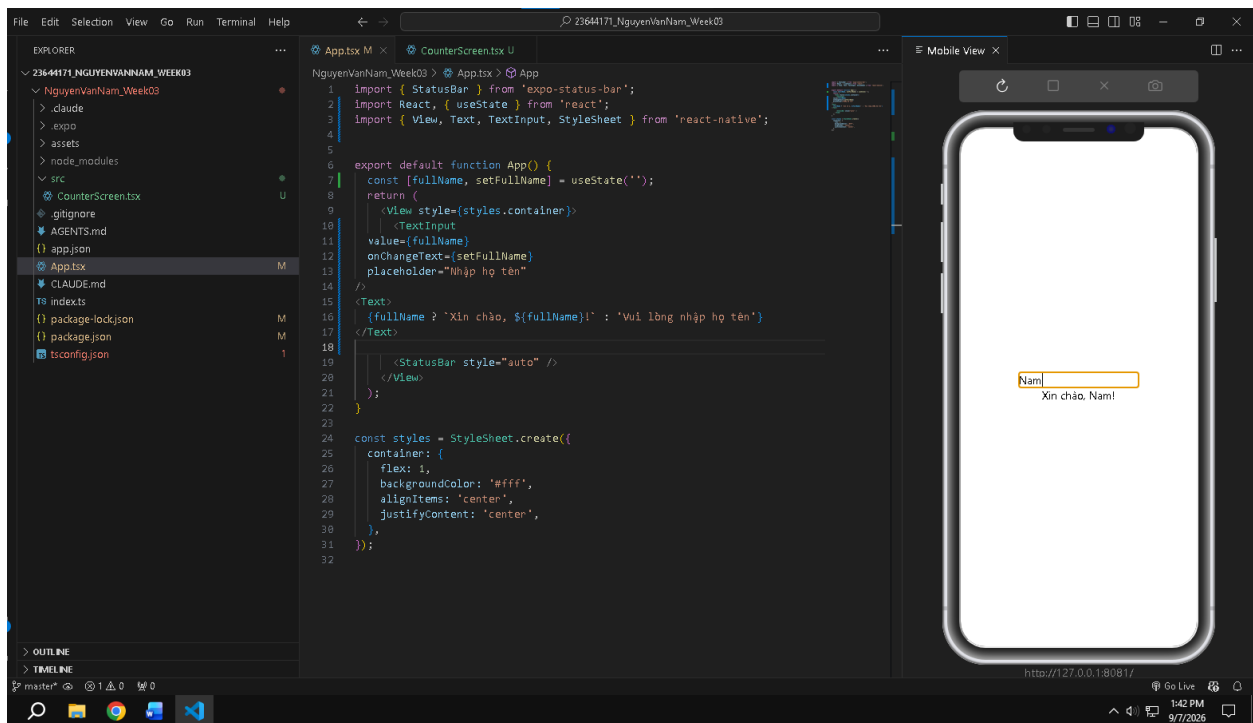
Bước 2: Khởi tạo trạng thái lưu họ tên.

```
export default function App() {
  const [fullName, setFullName] = useState('');
```

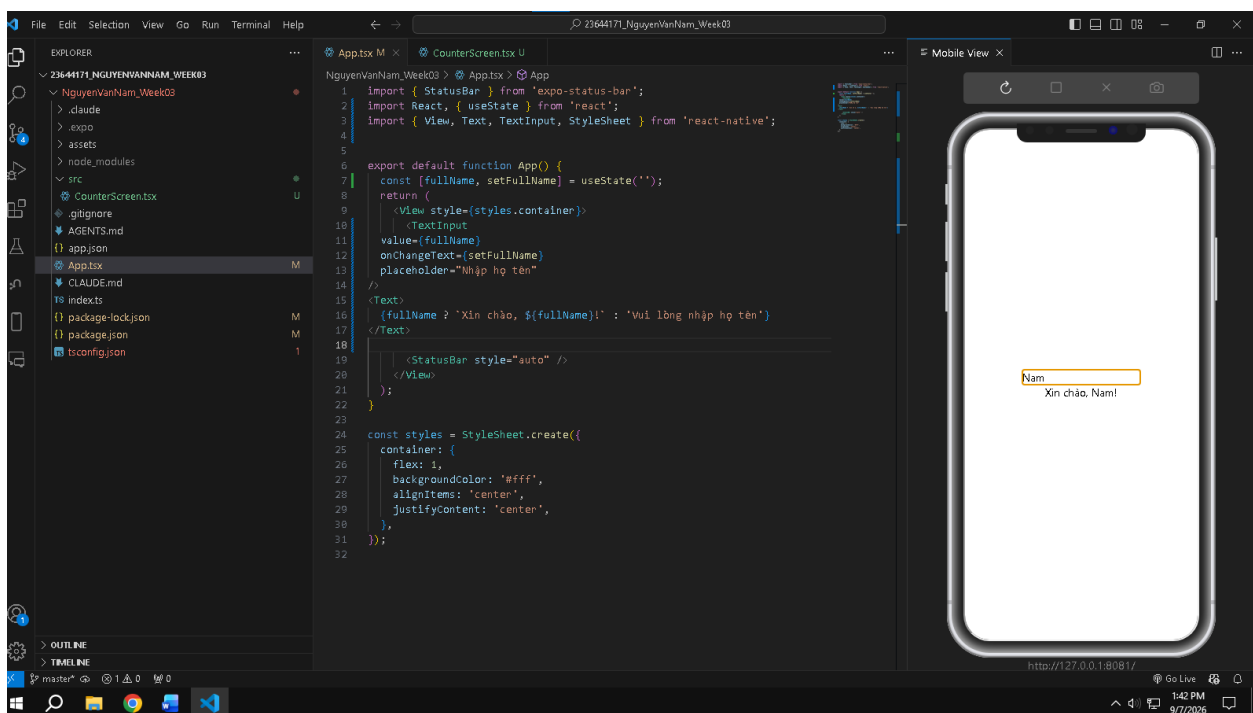
Bước 3: Tạo `TextInput` và cập nhật trạng thái khi người dùng nhập.

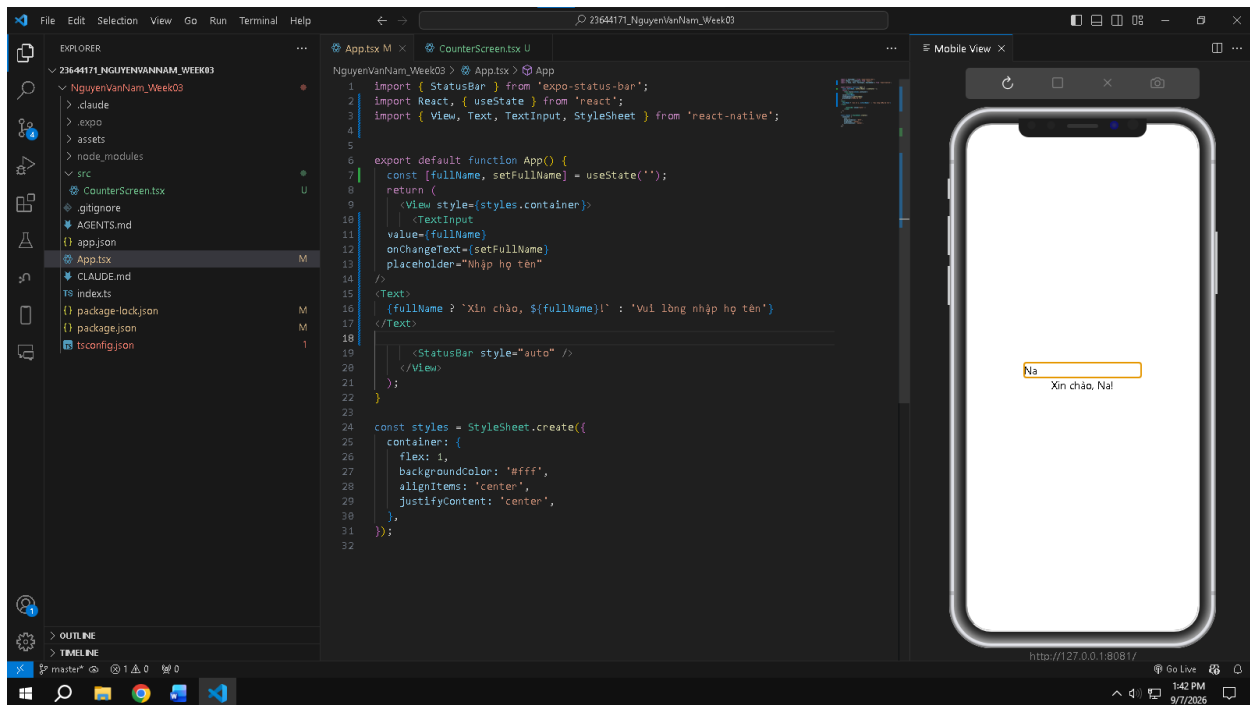


Bước 4: Hiển thị lời chào.



Bước 5: Chạy ứng dụng và kiểm tra giao diện khi nhập hoặc xóa nội dung.





Sản phẩm mong đợi

- Ô nhập hiển thị đúng nội dung người dùng nhập.
- Lời chào thay đổi ngay khi trạng thái thay đổi.
- Khi ô nhập rỗng, ứng dụng hiển thị lời nhắc.

Câu hỏi mở rộng

1. Bổ sung trạng thái để lưu tuổi.

```

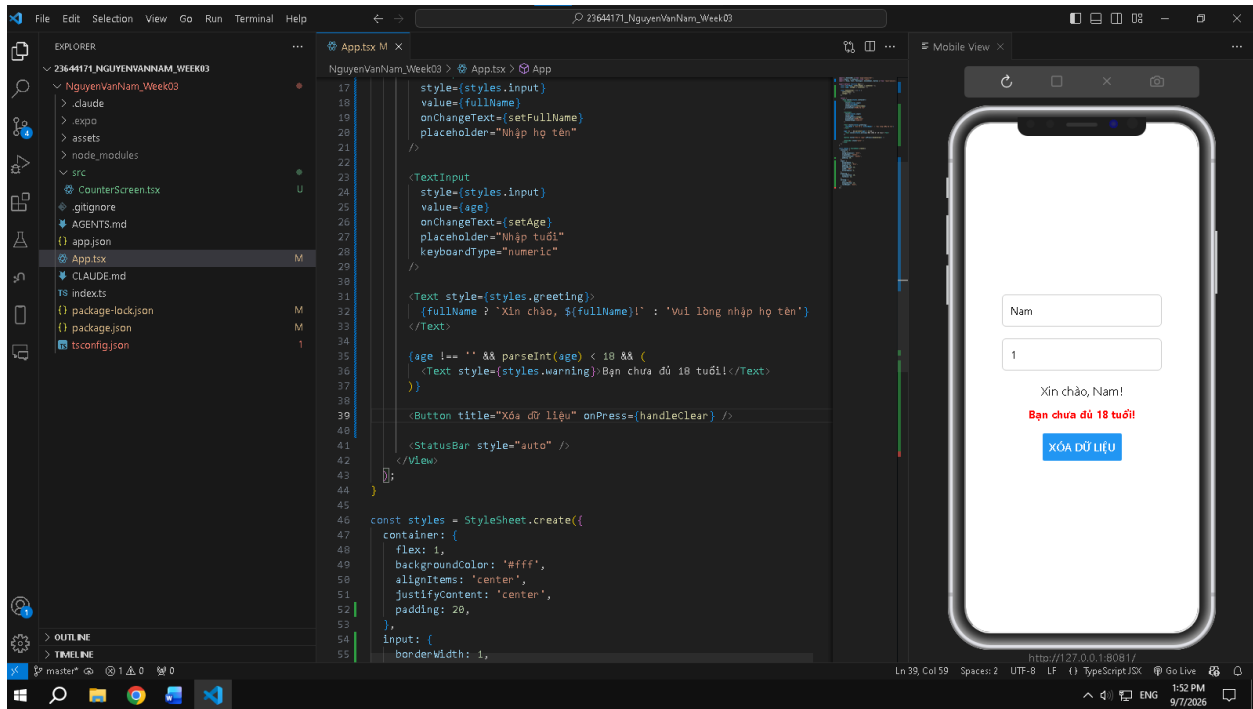
export default function App() {
  const [fullName, setFullName] = useState('');
  const [age, setAge] = useState('');

```

2. Hiển thị thông báo nếu tuổi nhỏ hơn 18.



3. Thêm nút để xóa toàn bộ dữ liệu đã nhập.



3.1.2. useEffect

Ví dụ áp dụng: Đồng hồ đếm giây

3. Bài tập thực hành: Theo dõi trạng thái kết nối giả lập

Yêu cầu: Tạo một công tắc kết nối. Mỗi khi trạng thái thay đổi, cập nhật thông báo trên màn hình.

Hướng dẫn từng bước

Bước 1: Tạo hai trạng thái.

```
export default function App() {  
  const [isConnected, setIsConnected] = useState(false);  
  const [message, setMessage] = useState('Chưa kết nối');
```

Bước 2: Dùng useEffect theo dõi isConnected.

```
useEffect(() => {
  if (isConnected) {
    setMessage('Thiết bị đã kết nối');
  } else {
    setMessage('Thiết bị đã ngắt kết nối');
  }
}, [isConnected]);
```

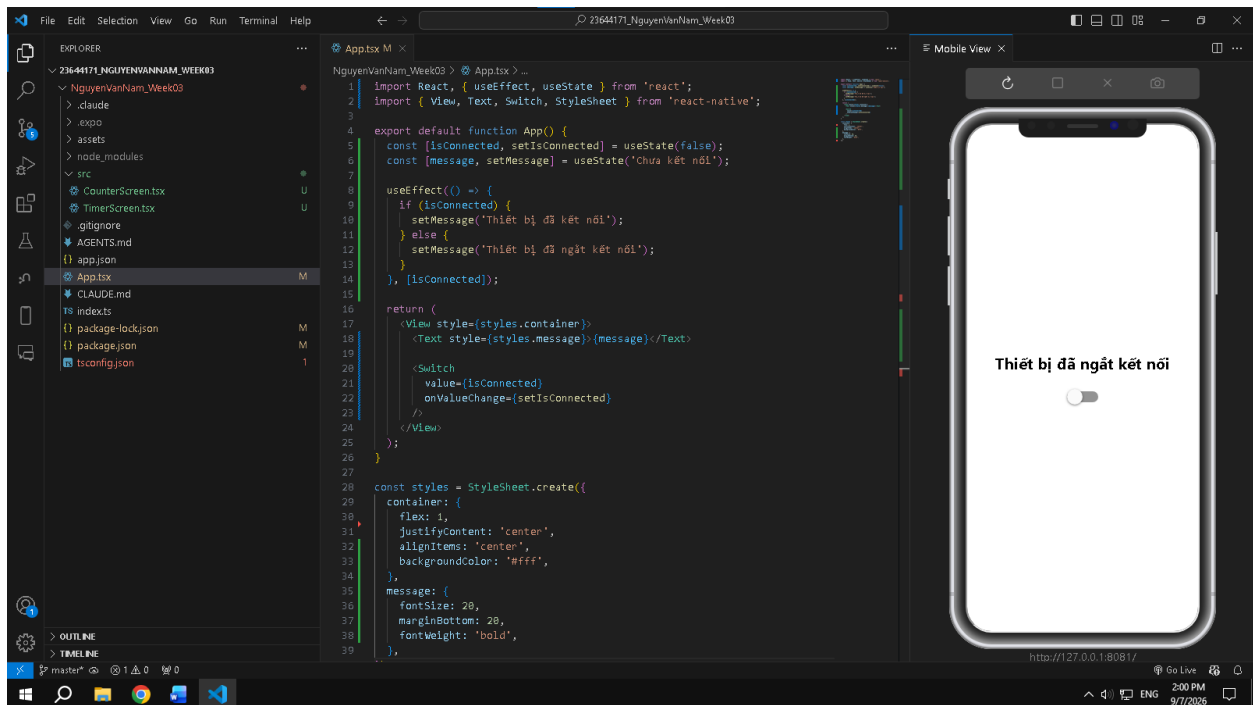
Bước 3: Tạo Switch để thay đổi trạng thái.

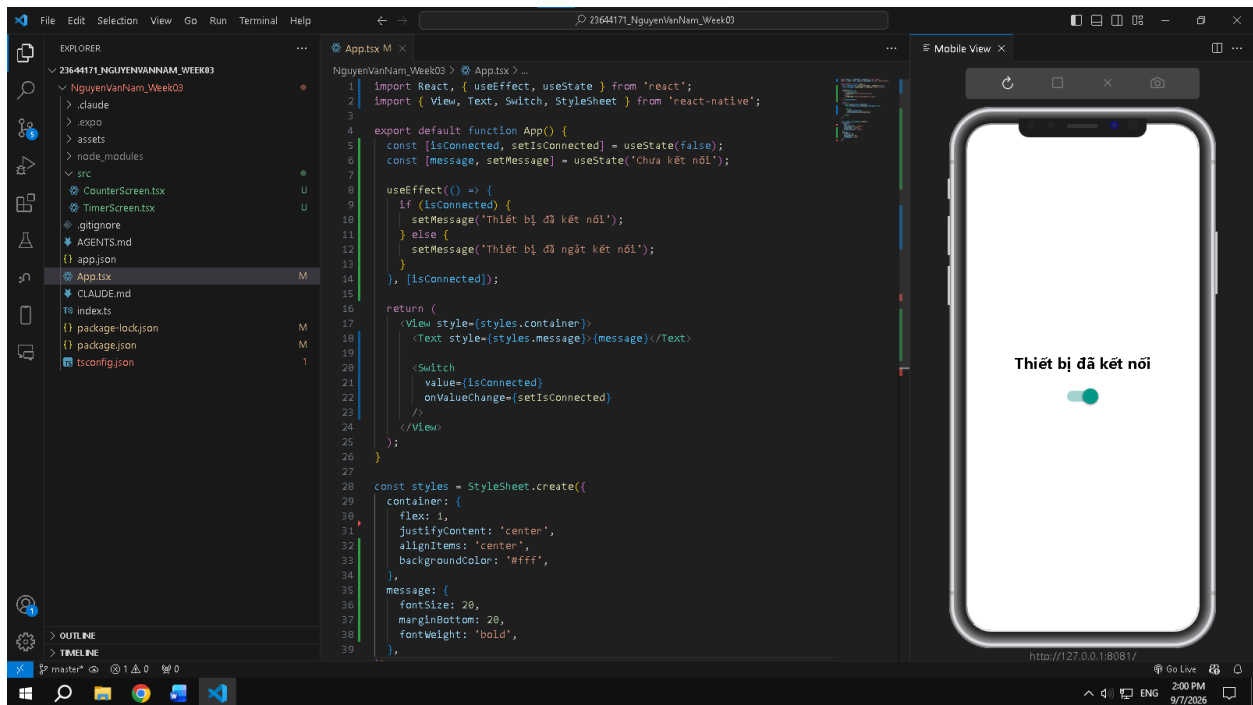
```
<Switch
  value={isConnected}
  onValueChange={setIsConnected}
/>
```

Bước 4: Hiển thị thông báo.

```
<View style={styles.container}>
  <Text style={styles.message}>{message}</Text>
</View>
```

Bước 5: Bật và tắt công tắc, sau đó quan sát thông báo.





Sản phẩm mong đợi

- Bật công tắc: hiển thị “Thiết bị đã kết nối”.
- Tắt công tắc: hiển thị “Thiết bị đã ngắt kết nối”.
- Effect chỉ chạy lại khi `isConnected` thay đổi.

3.1.3. useContext

Bài tập thực hành: Chia sẻ thông tin người dùng

Yêu cầu: Cung cấp tên người dùng từ component App và hiển thị nó trong ProfileScreen.

Hướng dẫn từng bước

Bước 1: Tạo context.

```

venVanNam_Week03 > src > exercises > Bai3_UseContext.tsx > ProfileScreen
import React, { createContext, useContext, useState } from 'react';
import { View, Text, StyleSheet, Button } from 'react-native';

const UserContext = createContext<any>(null);

```

Bước 2: Bao component con bằng Provider.


```
export default function Bai3_UseContext() {  
  return (  
    <UserContext.Provider value={{ name: 'Nguyễn Văn Nam' }}>  
      <div style={styles.container}>  
        <h1>Chào mừng bạn!</h1>  
      </div>  
    </UserContext.Provider>  
  );  
}
```

Bước 3: Nhận dữ liệu trong ProfileScreen.

```
function ProfileScreen() {  
  const user = useContext(UserContext);  
  return (  
    <div style={styles.container}>  
      <h2>Thông tin người dùng</h2>  
      <p>Tên: {user.name}</p>  
    </div>  
  );  
}
```

Bước 4: Hiển thị tên người dùng.



Bước 5: Thay đổi giá trị `name` ở `Provider` và quan sát kết quả.

Sản phẩm mong đợi

- `ProfileScreen` nhận được tên người dùng.
- Không cần truyền `name` trực tiếp qua props.
- Khi giá trị của `Provider` thay đổi, component sử dụng context được cập nhật.

3.1.4. useReducer

Bài tập thực hành: Quản lý form đăng nhập

Yêu cầu: Dùng `useReducer` để quản lý email, mật khẩu và thông báo lỗi.

Hướng dẫn từng bước

Bước 1: Khai báo trạng thái ban đầu.

```
NguyenVanNam_Week03 > src > exercises > Bai4_UseReducer.tsx > ...
1  import React, { useReducer } from 'react';
2  import { View, Text, TextInput, Button, StyleSheet } from 'react-native';
3
4  const initialState = { email: '', password: '', error: '' };
5
```

Bước 2: Viết reducer.

```
nguyenVanNam_Week03 > src > exercises > Bai4_UseReducer.tsx > ...
1  import React, { useReducer } from 'react';
2  import { View, Text, TextInput, Button, StyleSheet } from 'react-native';
3
4  const initialState = { email: '', password: '', error: '' };
5
6  function formReducer(state: any, action: any) {
7    switch (action.type) {
8      case 'SET_EMAIL': return { ...state, email: action.payload, error: '' };
9      case 'SET_PASSWORD': return { ...state, password: action.payload, error: '' };
10     case 'SET_ERROR': return { ...state, error: action.payload };
11     case 'RESET': return initialState;
12     default: return state;
13   }
14 }
15
```

Bước 3: Khởi tạo reducer trong component.

```
export default function Bai4_UseReducer() {
  const [state, dispatch] = useReducer(formReducer, initialState);
}
```

Bước 4: Kết nối các ô nhập với reducer.

```

NguyenVanNam_Week03 > src > exercises > Bai4_UseReducer.tsx > ...
17  const [state, dispatch] = useReducer(formReducer, initialState);
18
19  const handleLogin = () => {
20    if (!state.email || !state.password) {
21      dispatch({ type: 'SET_ERROR', payload: 'Vui lòng nhập đầy đủ thông tin' });
22      return;
23    }
24    dispatch({ type: 'SET_ERROR', payload: 'Đăng nhập thành công!' });
25  };
26
27  return (
28    <View style={styles.container}>
29      <Text style={styles.header}>Bài 3.1.4: useReducer</Text>
30
31      <TextInput
32        style={styles.input}
33        value={state.email}
34        onChangeText={text => dispatch({ type: 'SET_EMAIL', payload: text })}
35        placeholder="Email"
36      />
37      <TextInput
38        style={styles.input}
39        value={state.password}
40        onChangeText={text => dispatch({ type: 'SET_PASSWORD', payload: text })}
41        placeholder="Mật khẩu"
42        secureTextEntry
43      />
44
45      {state.error ? <Text style={styles.error}>{state.error}</Text> : null}
46
47      <View style={styles.buttonGroup}>
48        <Button title="Đăng nhập" onPress={handleLogin} />
49        <View style={{ height: 10 }} />
50        <Button title="Đặt lại" color="red" onPress={() => dispatch({ type: 'RESET' })} /
51      </View>
52    </View>
53  );
54 }
55

```

Bước 5: Kiểm tra dữ liệu khi người dùng nhấn đăng nhập.

```

const handleLogin = () => {
  if (!state.email || !state.password) {
    dispatch({ type: 'SET_ERROR', payload: 'Vui lòng nhập đầy đủ thông tin' });
    return;
  }
  dispatch({ type: 'SET_ERROR', payload: '' });
};

```

Bước 6: Hiển thị lỗi và thêm nút đặt lại.

```

    })

    {state.error ? <Text style={styles.error}>{state.error}</Text> : null}

    <View style={styles.buttonGroup}>
      <Button title="Đăng nhập" onPress={handleLogin} />
      <View style={{ height: 10 }} />
      <Button title="Đặt lại" color="red" onPress={() => dispatch({ type: 'RESET' })} />
    </View>
  </View>
)

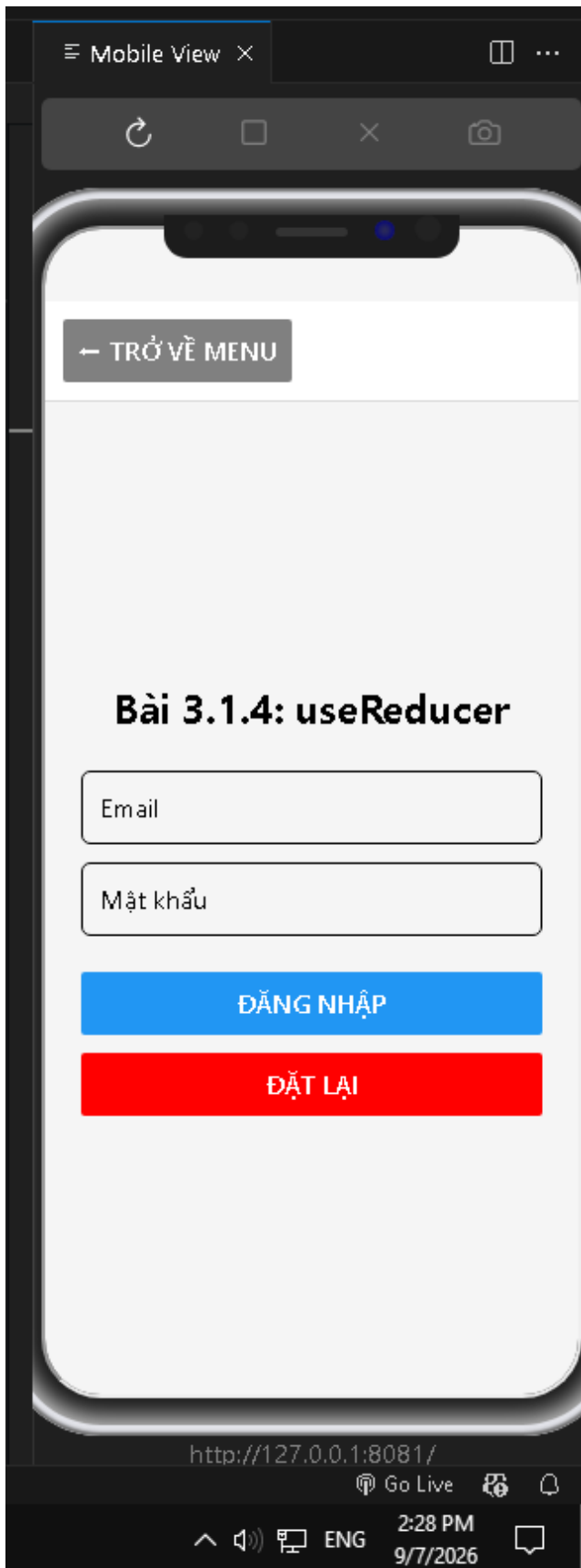
```

Sản phẩm mong đợi

- Email và mật khẩu được lưu trong cùng một state.
- Bỏ trống dữ liệu sẽ xuất hiện thông báo lỗi.



- Nút “Đặt lại” đưa form về trạng thái ban đầu.



3.1.5. useMemo và useCallback

Bài tập thực hành: Tìm kiếm và tính tổng sản phẩm

Yêu cầu: Tạo danh sách sản phẩm, lọc theo tên và tính tổng giá của các sản phẩm đang hiển thị.

Hướng dẫn từng bước

Bước 1: Tạo danh sách sản phẩm.

```
Bai5_UseMemo.tsx U X
NguyenVanNam_Week03 > src > exercises > Bai5_UseMemo.tsx > ...
1 import React, { useState, useMemo, useCallback } from 'react';
2 import { View, Text, TextInput, Button, FlatList, StyleSheet } from 'react-native';
3
4 const products = [
5   { id: '1', name: 'Áo thun', price: 200000 },
6   { id: '2', name: 'Quần jean', price: 450000 },
7   { id: '3', name: 'Giày thể thao', price: 800000 },
8 ];
9
```

Bước 2: Tạo trạng thái từ khóa tìm kiếm.

```
export default function Bai5_UseMemo() {
  const [keyword, setKeyword] = useState('');
  const filteredProducts = useMemo(() => {
    return products.filter(product =>
      product.name.toLowerCase().includes(keyword.toLowerCase())
    );
  }, [keyword]);
}
```

Bước 3: Dùng useMemo để lọc sản phẩm.

```
const filteredProducts = useMemo(() => {
  return products.filter(product =>
    product.name.toLowerCase().includes(keyword.toLowerCase())
  );
}, [keyword]);
```

Bước 4: Dùng useMemo để tính tổng giá.

```
const totalPrice = useMemo(() => {
  return filteredProducts.reduce((total, product) => total + product.price, 0);
}, [filteredProducts]);
```

Bước 5: Tạo hàm chọn sản phẩm bằng `useCallback`.

```
const handleSelect = useCallback((product: any) => {  
  alert(`Đã chọn: ${product.name}`);  
}, []);
```

Bước 6: Hiển thị danh sách bằng `FlatList`.

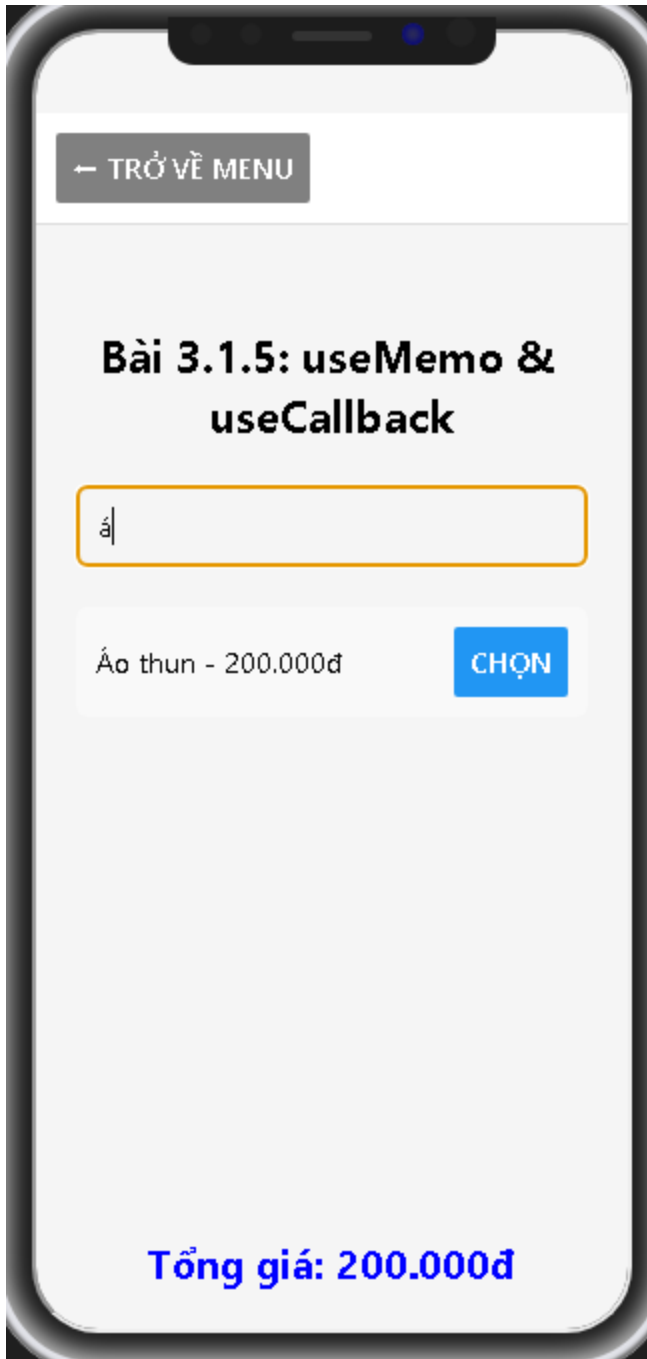
```
<FlatList  
  data={filteredProducts}  
  keyExtractor={item => item.id}  
  renderItem={({ item }) => (  
    <View style={styles.item}>  
      <Text style={{flex: 1}}>{item.name} - {item.price.toLocaleString('vi-VN')}đ</Text>  
      <Button title="Chọn" onPress={() => handleSelect(item)} />  
    </View>  
  )}  
/>
```

Bước 7: Hiển thị tổng giá.

```
<Text style={styles.total}>  
  Tổng giá: {totalPrice.toLocaleString('vi-VN')}đ  
</Text>
```

Sản phẩm mong đợi

- Danh sách thay đổi theo từ khóa.



- Tổng giá được tính từ các sản phẩm đang hiển thị.



- Khi từ khóa không thay đổi, phép tính lọc không cần thực hiện lại.
- Hàm `handleSelect` giữ nguyên tham chiếu giữa các lần render.

Bài tập tổng hợp cuối chương

Xây dựng ứng dụng quản lý công việc cá nhân

Yêu cầu

Ứng dụng có các chức năng:

- Thêm công việc.
- Đánh dấu công việc đã hoàn thành.
- Xóa công việc.
- Lọc công việc theo từ khóa.
- Chuyển đổi giao diện sáng và tối.
- Hiển thị số công việc chưa hoàn thành.

Hook cần sử dụng

- `useState`: quản lý nội dung ô nhập và từ khóa tìm kiếm.
- `useEffect`: lưu hoặc ghi nhận sự thay đổi của danh sách.
- `useContext`: chia sẻ chủ đề sáng/tối.
- `useReducer`: quản lý danh sách và các hành động thêm, xóa, hoàn thành.
- `useMemo`: lọc danh sách và tính số công việc chưa hoàn thành.
- `useCallback`: tạo các hàm xử lý truyền xuống component con.

Hướng dẫn thực hiện

Bước 1: Xây dựng cấu trúc dữ liệu của một công việc.

```
{
  id: '1',
  title: 'Học React Native',
  completed: false
}
```

Bước 2: Tạo reducer với các hành động:

- `ADD_TODO`
- `TOGGLE_TODO`
- `DELETE_TODO`

```
const handleAdd = useCallback(() => {
  if (taskInput.trim() === '') return;
  dispatch({ type: 'ADD_TODO', payload: taskInput });
  setTaskInput('');
}, [taskInput]);

const handleToggle = useCallback((id: string) => {
  dispatch({ type: 'TOGGLE_TODO', payload: id });
}, []);

const handleDelete = useCallback((id: string) => {
  dispatch({ type: 'DELETE_TODO', payload: id });
}, []);
```

Bước 3: Dùng `useState` quản lý ô nhập công việc và từ khóa.

```
const [taskInput, setTaskInput] = useState('');
const [searchKeyword, setSearchKeyword] = useState('');
```

Bước 4: Dùng `useContext` tạo chế độ sáng/tối cho toàn ứng dụng.

```
const ThemeContext = createContext<any>(null);
```

```
function TodoAppContent() {
  const { isDarkMode, toggleTheme } = useContext(ThemeContext);
  const [taskInput, setTaskInput] = useState('');
  const [searchKeyword, setSearchKeyword] = useState('');
```

Bước 5: Dùng `useMemo` để lọc công việc và đếm số việc chưa hoàn thành.

```
const filteredTodos = useMemo(() => {
  return todos.filter((todo: any) =>
    todo.title.toLowerCase().includes(searchKeyword.toLowerCase())
  );
}, [todos, searchKeyword]);

const incompleteCount = useMemo(() => {
  return todos.filter((todo: any) => !todo.completed).length;
}, [todos]);
```

Bước 6: Dùng `useCallback` cho các hàm đánh dấu hoàn thành và xóa công việc.

```
const handleToggle = useCallback((id: string) => {
  dispatch({ type: 'TOGGLE_TODO', payload: id });
}, []);

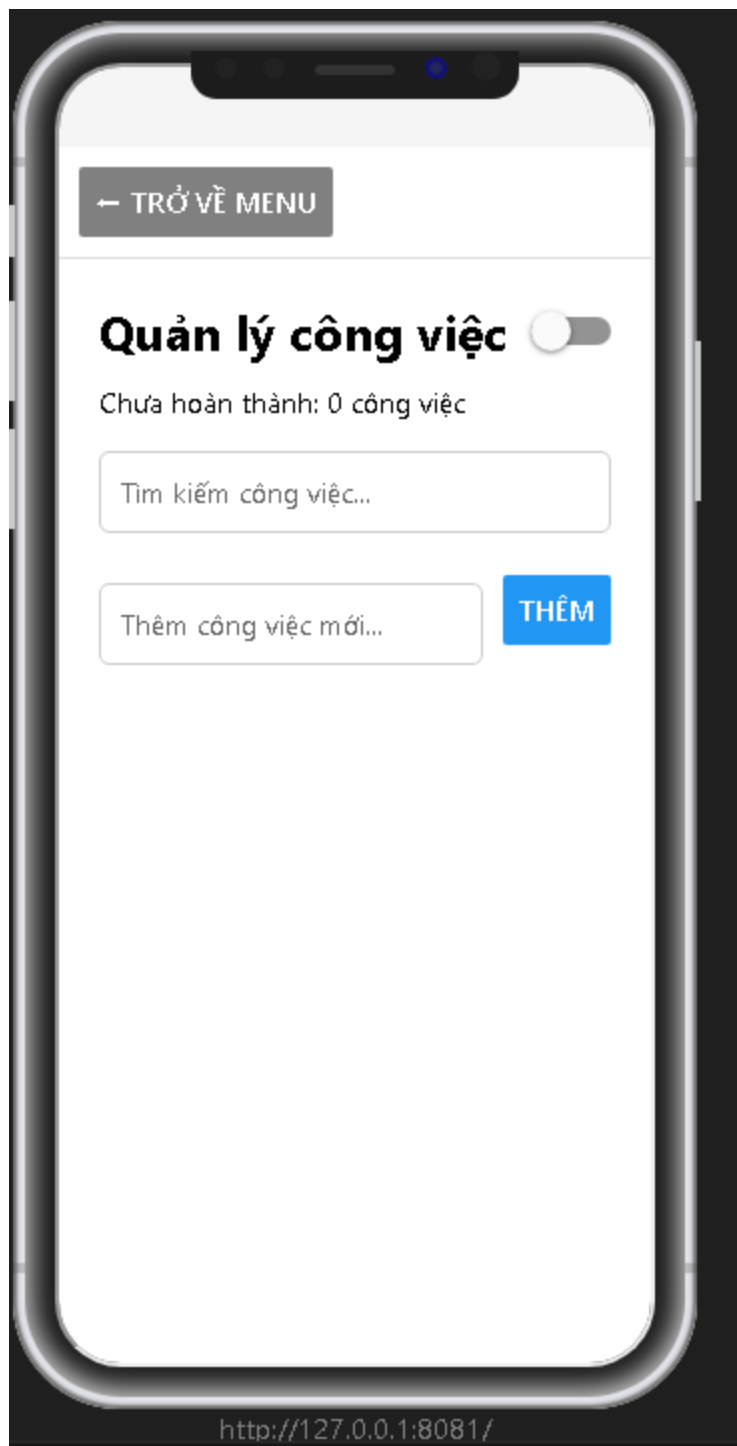
const handleDelete = useCallback((id: string) => {
  dispatch({ type: 'DELETE_TODO', payload: id });
}, []);
```

Bước 7: Dùng `useEffect` để theo dõi số lượng công việc:

```
useEffect(() => {
  console.log(`[Hệ thống] Danh sách hiện có ${todos.length} công việc`);
}, [todos.length]);
```

Bước 8: Kiểm thử đầy đủ các chức năng và các trường hợp danh sách rỗng.

Giao diện màn hình sáng:



Giao diện màn hình tối:



Thêm công việc:



← TRỞ VỀ MENU

Quản lý công việc ☐

Chưa hoàn thành: 1 công việc

THÊM

ABC

XÓA



Đánh dấu công việc:



← TRỞ VỀ MENU

Quản lý công việc ☐

Chưa hoàn thành: 0 công việc

THÊM

ABC

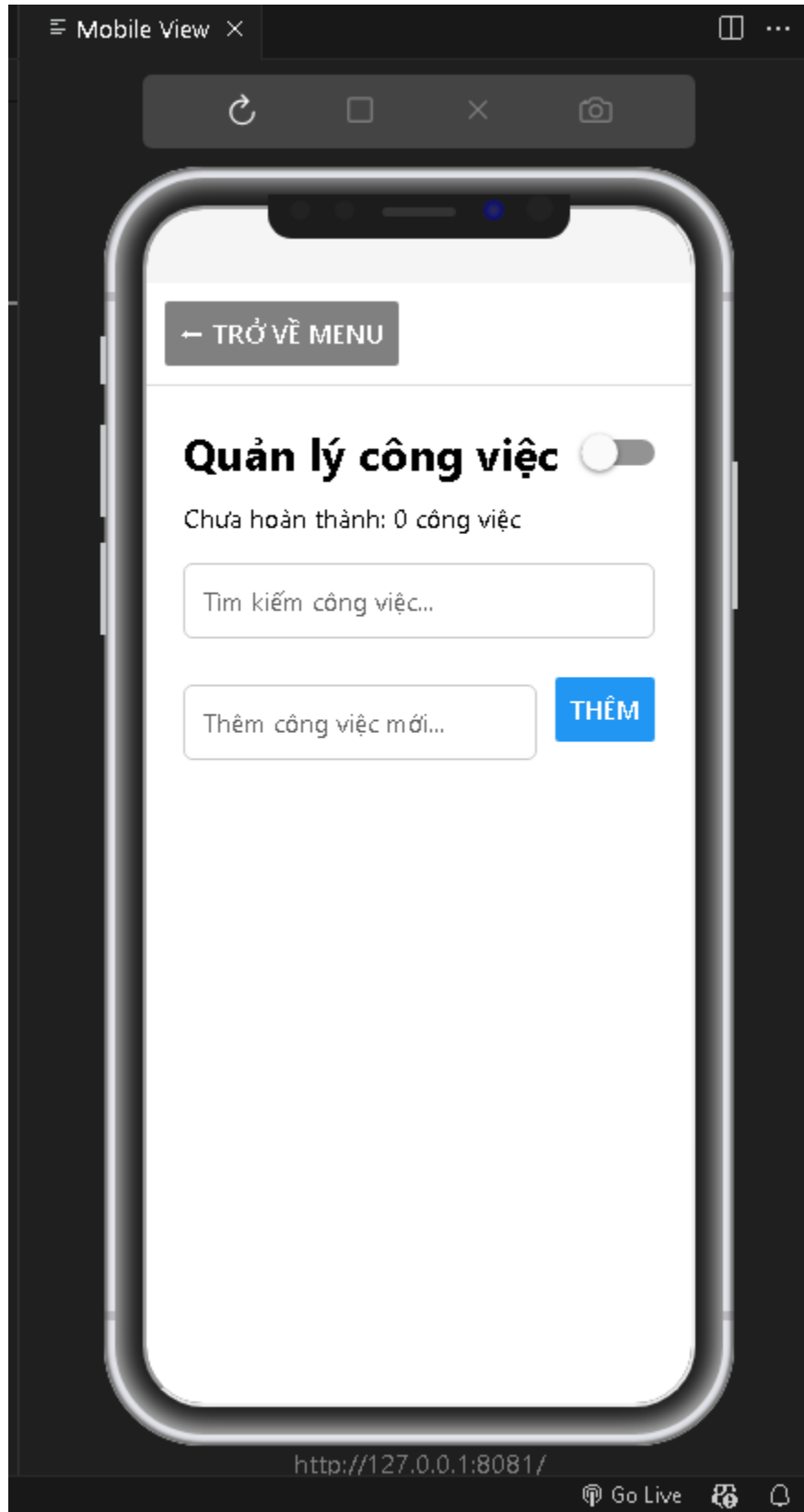
XÓA

http://127.0.0.1:8081/

Go Live



Xóa công việc:



Tiêu chí đánh giá

Nội dung	Điểm
Giao diện rõ ràng, dễ sử dụng	1
Thêm công việc đúng yêu cầu	1
Hoàn thành và xóa công việc	2
Sử dụng đúng <code>useReducer</code>	2
Sử dụng đúng <code>useContext</code>	1
Sử dụng hợp lý <code>useEffect</code>	1
Áp dụng <code>useMemo</code> và <code>useCallback</code>	1
Mã nguồn rõ ràng, có tổ chức	1
Tổng cộng	10

Câu hỏi ôn tập

1. Khi nào nên dùng `useState` thay vì `useReducer`?

Dùng `useState` khi dữ liệu đơn giản (chuỗi, số, boolean) hoặc các state hoạt động độc lập, ít liên quan đến nhau

2. Dependency array của `useEffect` có tác dụng gì?

Khi những biến bên trong Dependency array “[]” thay đổi thì mới chạy lại code bên trong `useEffect`

3. Tại sao effect sử dụng bộ đếm thời gian cần có hàm dọn dẹp?

Khi không có hàm dọn dẹp, bộ đếm đó vẫn cứ chạy trong background. Lâu dần sẽ gây giật lag và rò rỉ bộ nhớ

4. `useContext` giải quyết hạn chế nào của việc truyền props?

Gom dữ liệu lên 1 nơi lưu trữ chung, components nào sử dụng dữ liệu thì sẽ lấy dữ liệu ở nơi lưu trữ chung đó không phải truyền từ components này sang components khác để lấy dữ liệu

5. `useMemo` ghi nhớ giá trị hay hàm?

Ghi nhớ giá trị (kết quả sau khi chạy một phép tính nặng)

6. `useCallback` ghi nhớ giá trị hay hàm?

Ghi nhớ hàm (địa chỉ của function đó trong bộ nhớ)

7. Vì sao không nên sử dụng `useMemo` và `useCallback` cho mọi trường hợp?

Để chạy hai cái hook này, React phải tốn công theo dõi và so sánh xem dependency có đổi không. Với mấy phép tính nhẹ hay hàm quá đơn giản, việc "nhớ" khéo còn làm app chạy chậm hơn cả việc cứ để nó tính lại từ đầu

8. Reducer có được thay đổi trực tiếp state hiện tại không? Vì sao?

Không. Vì React so sánh bằng địa chỉ bộ nhớ. Nếu bạn sửa trực tiếp, địa chỉ không đổi, React sẽ không nhận ra sự thay đổi nên không thêm render lại giao diện. Bắt buộc phải trả về một state mới hoàn toàn (thường dùng dấu `...` để copy)